

NoSQL Databases

Francesco Pelacani
2025-2026



Document Databases

What is a Document Database?

A document database is a non-relational (NoSQL) storage system designed to manage, store, and retrieve semi-structured data (JSON) as documents.

Unlike rigid relational databases, they use a **flexible, schema-free** model where each document can have a unique structure, allowing for rapid development and easier scaling.



Features

- **Consistency** → Configured via replica sets
- **Transactions** → Atomicity at single-document level
- **Availability** → Master-slave replication to improve it
- **Query features** → Can query the data inside a document without needing to retrieve the whole document. QL expressed via JSON-like language.
- **Scaling** → Add more slaves to improve read capabilities.

Why is MongoDB CP in Cap Teorem?

MongoDB is classified as CP because, when a network partition occurs, it prioritizes data integrity over guaranteed responsiveness. While replica sets provide "high availability" in a general sense, they do not meet the strict definition of Availability as defined by the CAP theorem.

In the CAP theorem, *Availability* means that every request to a non-failing node must receive a successful (non-error) response. MongoDB breaks this during two specific scenarios:

- **Election Downtime:** When a primary node fails, the cluster must elect a new one. During this window (usually a few seconds), the system rejects all writes.
- **Lack of Quorum:** If a network partition splits the cluster such that no group has a majority (e.g., 2 nodes out of 5), MongoDB will not allow any node to become primary. It sacrifices the ability to write to prevent "split-brain" scenarios where two different primaries might accept conflicting data.

Why it is not AP?

An AP system, like Cassandra, allows any node to accept writes even if it can't talk to the rest of the cluster.

Conflict Risk: AP systems accept the risk of data divergence (different nodes having different versions of the same record) to stay online.

MongoDB's single-master architecture ensures there is only ever one source of truth for writes. It would rather return an error than allow a write that it cannot immediately guarantee as consistent across a majority of nodes.

Usage

Suitable Use Cases

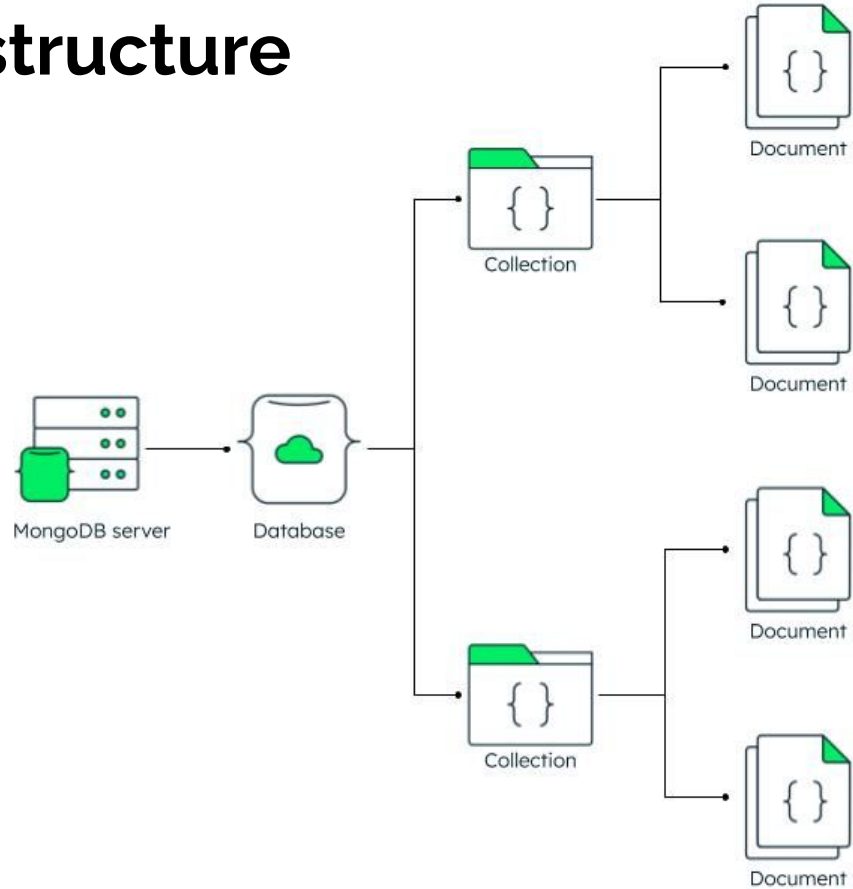
1. Event Logging
2. Content Management Systems
3. E-Commerce application

When Not to Use

1. Complex Transactions Spanning Different Operations
2. Queries against Varying Aggregate Structure

MongoDB

MongoDB structure



A sample Document



```
{
  "_id": "5cf0029cafff5056591b0ce7d",
  "firstname": "Jane",
  "lastname": "Wu",
  "address": {
    "street": "1 Circle Rd",
    "city": "Los Angeles",
    "state": "CA",
    "zip": "90404"
  },
  "hobbies": ["surfing", "coding"]
}
```

BSON (I)

BSON is a binary serialization format used to store documents and make remote procedure calls in MongoDB.

Contains extensions that allow representation of data types that are not part of the JSON spec (Date, Binary, ..).

BSON (II)

The three main characteristics of BSON format:

1. **Lightweight** → Keeping spatial overhead to a minimum is important for any data representation format, especially when used over the network.
2. **Traversable** → BSON is designed to be traversed easily. This is a vital property in its role as the primary data representation for MongoDB.
3. **Efficient** → Encoding data to BSON and decoding from BSON can be performed very quickly.

Document Characteristics

Field names are **strings**, must be unique and should be descriptive about their contents.

The **values** of a field can be of various data types: Strings, Numbers, Booleans, Arrays, Document, Dates (timestamp), ObjectId (*_id*).

Are stored, and transmitted over the network, in a format called BSON (Binary JSON).

Its flexible schema allow to handle **polymorphic data**.

Max document size of 16MB, maximum of 100 levels of nesting.

Entities and Attributes

Entities are the single piece of information stored in MongoDB, they are unique and independent and represent distinct objects or concept.

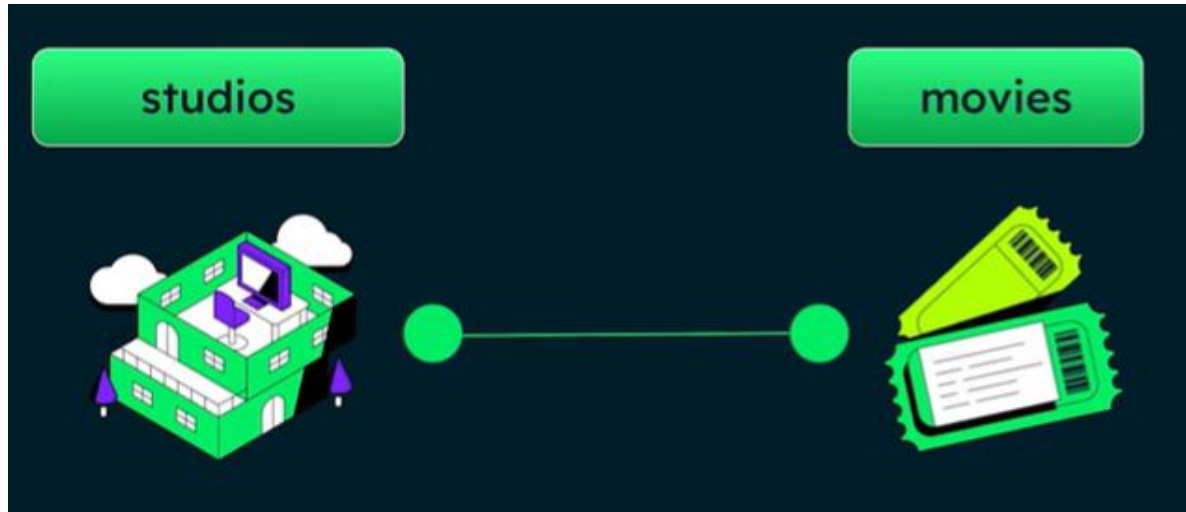
Usually entities are represented by **documents** and grouped in **collections**.

Attributes provide specific information about the entity it belongs to.

1-1 Relationships

One entity is related to one other entity.

E.g. A movie is released by a single studio.



How to model relationships (I)

For 1-1 relationships we can **embed** the related data directly within the parent document.

Related data is kept together and simplifies queries.

```
{
  _id: ObjectId('573a1395f29313caabce2498'),
  title: 'A Fistful of Dollars',
  genres: [ 'Action', 'Drama', 'Western' ],
  runtime: 99,
  rated: 'R',
  studio: 'Universal Pictures'
  ...
}
```

How to model relationships (II)

Also in 1-N relationships we can **embed** the related data directly within the parent document.

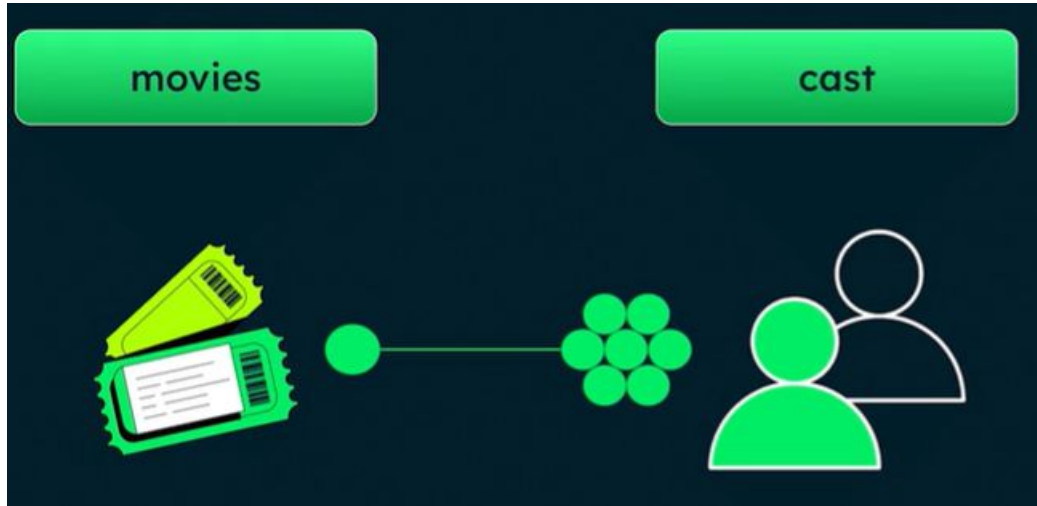
Related data is kept together and simplifies queries.

```
{
  _id: ObjectId('573a1395f29313caabce2498'),
  plot: 'A wandering gunfighter plays two rival
        families against each other in a town torn
        apart by greed, pride, and revenge.',
  genres: [ 'Action', 'Drama', 'Western' ],
  runtime: 99,
  rated: 'R',
  cast: [
    'Clint Eastwood',
    'Marianne Koch',
    'Gian Maria Volontè',
    'Wolfgang Lukschy'
  ],
  ...
  title: 'A Fistful of Dollars',
  ...
}
```

1-N Relationships

One entity is related to multiple other entities.

E.g. A movie as multiple cast members.



How to model relationships (III)

However if we maintain a large portion of data related to each entity on the many side, we can **reference** the cast members with an array of referenced IDs that point to the appropriate document in the collection.

```
{
  _id: ObjectId('573a1395f29313caabce2498'),
  plot: 'A wandering gunfighter plays two rival
    families against each other in a town torn
    apart by greed, pride, and revenge.',
  genres: [ 'Action', 'Drama', 'Western' ],
  runtime: 99,
  rated: 'R',
  cast: [
    '66b64efd547b38844f2f0780',
    '66b64f17f97ccd9bd0385ea7',
    '66b64f23c7259012041f0a85',
    '66b6506777e68c490b841f2e'
  ],
  ...
  title: 'A Fistful of Dollars',
  ...
}
```

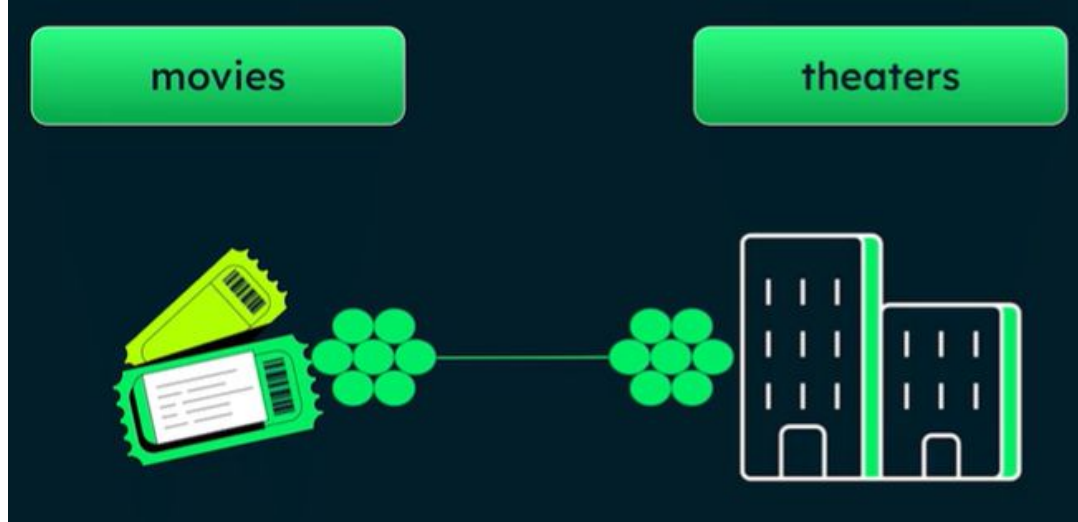


```
{
  _id: ObjectId('66b64efd547b38844f2f0780'),
  name: 'Clint Eastwood',
  birth_name: 'Clint Eastwood Jr.',
  birth: ISODate('1930-05-31T00:00:00.000Z'),
  death: null,
  movies: [
    '573a1396f29313caabce49bf',
    '573a1393f29313caabcde458',
    '573a1394f29313caabcde87e',
    '573a13a9f29313caabd1f81d',
    ...
  ],
}
```

N-N Relationships

Multiple entities are related to multiple other entities.

E.g. A movie is usually shown in different cinema, and a cinema can show many different movies at a time.



How to model relationships (IV)

What about N-N relationships?

We can use either embeddings or referencing, it depends on our application and requirements.

```
{
  _id: ObjectId('573a1397f29313caabce7b71'),
  plot: 'A young boy accidentally joins a band
of dwarves as they jump from era to era looking
for treasure to steal.',
  genres: [ 'Adventure', 'Comedy', 'Fantasy'
],
  runtime: 110,
  rated: 'PG',
  cast: [
    'John Cleese',
    'Sean Connery',
    'Shelley Duvall',
    'Katherine Helmond'
  ],
  ...
  title: 'Time Bandits',
  ...
  theaters: [
    "Cinepolis Ocean Ranch 7",
    "Reading Cinemas at Cal Oaks Plaza",
    "AMC UTC 14",
    "Regal Edwards Cinemas"
  ],
  ...
  num_mflix_comments: 0
}

{
  _id: ObjectId('66b52903b81528dac869cdeb'),
  theaterId: 1138,
  theater_name: 'Regal Edwards Cinemas,
Temecula, CA'
  location: {
    address: {
      street1: '40750 Winchester Rd',
      city: 'Temecula',
      state: 'CA',
      zipcode: '92591'
    },
    geo: { type: 'Point', coordinates: [
      -117.15186, 33.52502, ] }
  },
  movies_showing: [
    'Time Bandits',
    'Deadpool',
    'Amadeus',
    'Snatch'
  ]
}
```

How to model relationships (V)

If regularly need to access each data independently from the other, we could replace the embedded arrays of literals with references to objects.

```
{
  "_id": ObjectId('573a1397f29313caabce7b71'),
  plot: 'A young boy accidentally joins a band
of dwarves as they jump from era to era looking
for treasure to steal.',
  genres: [ 'Adventure', 'Comedy', 'Fantasy'
],
  runtime: 110,
  rated: 'PG',
  cast: [
    'John Cleese',
    'Sean Connery',
    'Shelley Duvall',
    'Katherine Helmond'
  ],
  ...
  title: 'Time Bandits',
  ...
  theaters: [
    '66b52903b81528dac869cdeb',
    '66b52866d11b8eb04af005b1',
    '66b52871a2710d37429d0272',
    '66b5287ccb0aeb243d4eb310'
  ],
  ...
  num_mflix_comments: 0
}

{
  "_id": ObjectId('66b52903b81528dac869cdeb'),
  theaterId: 1138,
  theater_name: 'Regal Edwards Cinemas,
Temecula, CA'
  location: {
    address: {
      street1: '40750 Winchester Rd',
      city: 'Temecula',
      state: 'CA',
      zipcode: '92591'
    },
    geo: { type: 'Point', coordinates: [
      -117.15186, 33.52502, ] }
  },
  movies_showing: [
    '573a1397f29313caabce7b71',
    '573a1398f29313caabce9091',
    '573a1398f29313caabce8f74',
    '573a1398f29313caabcea665'
  ]
}
```

Embedding vs Referencing

Embedding is ideal when:

- data is accessed as a single unit
- dataset don't change often
- there's a need to minimize database operations and joins to retrieve related data

Referencing is ideal when:

- data is accessed independently
- documents tends to grow in size

Work with data in MongoDB

Insert a Document

Use *insertOne()* to insert a document into a collection.

Within the parentheses of *insertOne()*, include an object that contains the document data. Here's an example:

```
db.grades.insertOne({  
  student_id: 654321,  
  products: [  
    {  
      type: "exam",  
      score: 90,  
    },  
    {  
      type: "homework",  
      score: 59,  
    },  
  ],  
  class_id: 550,  
})
```

Retrieve a Document

When given equality with an `_id` field, the *find()* command will return the specified document that matches the `_id`.

```
db.zips.find({ _id: ObjectId("5c8eccc1caa187d17ca6ed16") })
```

Use the *\$in* operator to select documents where the value of a field equals any value in the specified array.

```
db.zips.find({ city: { $in: ["PHOENIX", "CHICAGO"] } })
```

Other comparison operators

The *\$gt* operator matches documents with a field greater than the given value.

```
db.sales.find({ "items.price": { $gt: 50}})
```

The *\$lt* operator matches documents with a field less than the given value.

```
db.sales.find({ "items.price": { $lt: 50}})
```

The *\$lte* operator matches documents with a field less than or equal to the given value.

```
db.sales.find({ "items.price": { $lte: 65}})
```

The *\$gte* operator matches documents with a field greater than or equal to the given value

```
db.sales.find({ "items.price": { $gte: 65}})
```

Replace a Document

To replace documents in MongoDB, we use the *replaceOne()* method. The *replaceOne()* method takes the following parameters:

- **filter**: A query that matches the document to replace.
- **replacement**: The new document to replace the old one with.

```
db.books.replaceOne(  
  {  
    _id: ObjectId("6282afeb441a74a98dbbec4e"),  
  },  
  {  
    title: "Data Science Fundamentals for Python and MongoDB",  
    isbn: "1484235967",  
    publishedDate: new Date("2018-5-10"),  
    thumbnailUrl:  
      "https://m.media-amazon.com/images/I/71opmUBc2wL._AC_UY218_.jpg",  
    authors: ["David Paper"],  
    categories: ["Data Science"],  
  }  
)
```

Update a Document (I)

The *updateOne()* method accepts a filter document, an update document with a specified action/operator, and an optional option object.

The *\$set* operator replaces the value of a field with the specified value

```
db.podcasts.updateOne(
  {
    _id: ObjectId("5e8f8f8f8f8f8f8f8f8f8f8f"),
  },
  {
    $set: {
      subscribers: 98562,
    },
  }
)
```

Update a Document (II)

The *upsert* option creates a new document if no documents match the filtered criteria.

```
db.podcasts.updateOne(
  { title: "The Developer Hub" },
  { $set: { topics: ["databases", "MongoDB"] } },
  { upsert: true }
)
```

The *\$push* operator adds a new value to an array field.

```
db.podcasts.updateOne(
  { id: ObjectId("5e8f8f8f8f8f8f8f8f8f8f8f") },
  { $push: { hosts: "Nic Raboy" } }
)
```

Delete a Document

To delete documents, use the *deleteOne()* or *deleteMany()* methods. Both methods accept a filter document and an options object.

```
db.podcasts.deleteOne({ _id: ObjectId("6282c9862acb966e76bbf20a") })
```

```
db.podcasts.deleteMany({category: "crime"})
```

Practice - I

practice

[Complete the exercise on Github](#)

MongoDB Transactions

ACID Transactions

A group of operations that will be completed as a unit or not at all, ensuring database safety and consistency.

Atomicity → all operation will either succeed or fail together

Consistency → all changes made by operations are consistent with database constraints

Isolation → multiple transaction happening at the same time do not affect each other

Durability → all changes withing a transactions will persist, no matter what

ACID Transactions in MongoDB

Single-document operations are already atomic in MongoDB.

```
db.products.updateOne(  
  { _id: 100 },  
  {  
    $set: {  
      quantity: 500,  
      details: {  
        model: "2600",  
        make: "Fashionaires"  
      },  
      tags: ["coats", "outerwear", "clothing"]  
    }  
  }  
)
```

ACID Transactions in MongoDB

Multi-document operations are not inherently atomic and do require extra steps to become ACID compliant.

MongoDB have to lock all documents involved in a transaction, affecting latency and performances.

A transaction has a max runtime of less than 1 minute after the first write in it.

Practice - II

practice

[Practice ACID Transaction with this script](#)

Extra MongoDB Material

[MongoDB University](#)